

Lecture 9

Software Verification and Validation

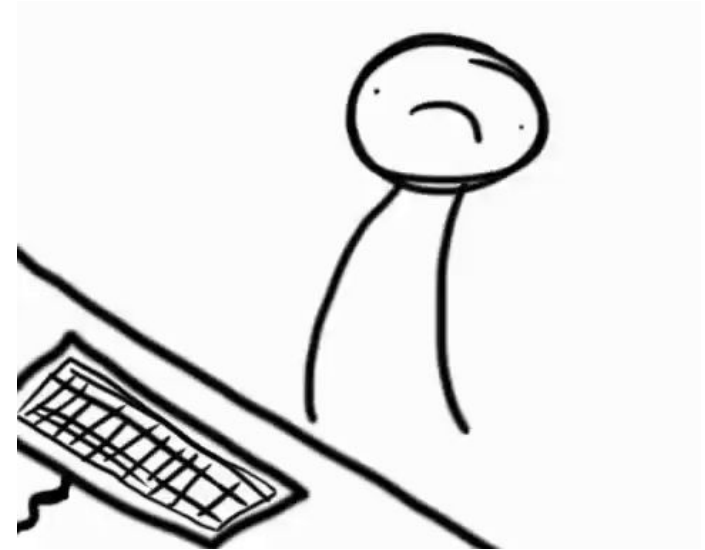
Automated Test Case Generation

Why do we test?

- Find bugs,
- Increase quality,
- Reducing software risk,
- Improving maintainability,
- Supporting teamwork,
- Enabling continuous delivery,
- Protecting users.

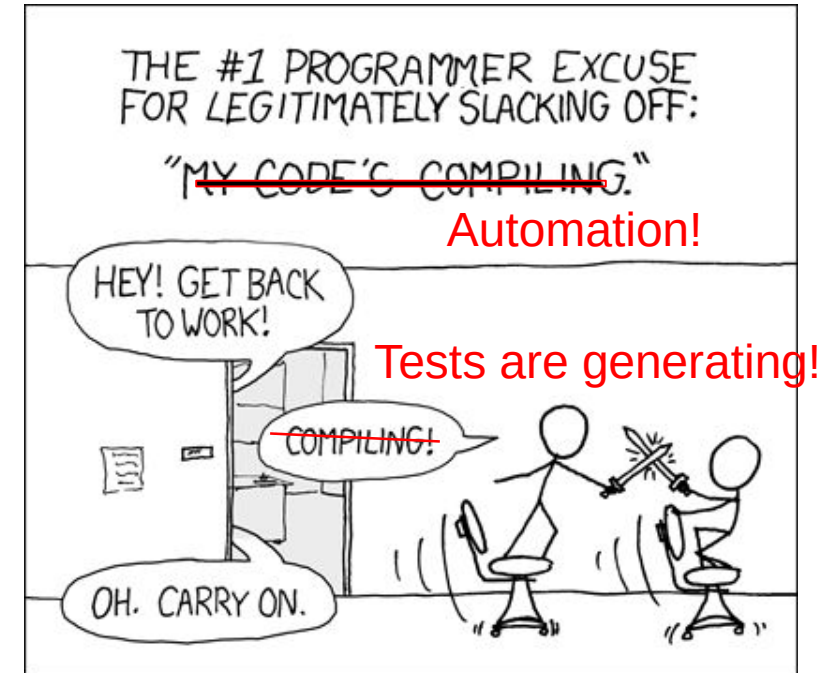
Automating Test Creation

- Testing is invaluable, but expensive.
- We test for *many* purposes.
- Near-infinite number of possible tests we could try.
- Hard to achieve meaningful volume.



Automation of Test Creation

- Relieve cost by automating test creation.
 - Repetitive tasks that do not **need** human attention.
- **Generate test input.**
 - Just need to add assertions.
 - (Or check for crashes, memory leaks, other problems that can be measured automatically.)



“Let machines do repetitive work.”

Test Creation as a Search Problem

- Do you have a **goal** in mind when testing?
 - *Make the program crash, achieve code coverage, cover all 2-way interactions, ...*
- You are **searching** for a test suite that achieves that goal.
- Search-based test generation based on **guess-and-check** process.

Test Creation as a Search Problem

- Many testing goals can be measured:
 - How many exceptions were thrown?
 - How many representative output values were returned?
 - What percentage of lines of code were covered?
 - How diverse is our input?
- If goal can be measured, search can be automated.

Search-Based Test Generation

- **Make one or more guesses.**
 - Generate one or more individual test cases or full test suites.
- **Check whether goal is met.**
 - Score each guess.
- **Try until time runs out.**
 - Alter the solution based on feedback and try again!



Machine Learning & AI in ATCG(Automatic Test Case Generation)

Once the goal of the test case is defined and the testing methodology is selected, we come to the final part. ML algorithms can learn from past test data and execution results to:

- **Prioritize Test Cases:** Identify which test cases are more likely to uncover defects based on historical data.
- **Generate data-driven Tests:** Automatically generate test data with specific characteristics to target specific functionalities or edge cases.
- **Self-learn and Adapt:** As the software evolves, the ML algorithms can learn and adapt their test generation strategies for continuous improvement.

The Impact of Automatic Test Case Generation on Software Quality

1. Increased Test Coverage

Manual test case creation can be time-consuming and often focuses on well-trodden paths. ATCG tools, on the other hand, can explore a vast array of scenarios, including edge cases and complex interactions, leading to significantly higher test coverage.

2. Improved Efficiency and Reduced Costs

ATCG automates a repetitive and time-consuming task, freeing up valuable tester resources for more strategic testing activities. This translates to faster testing cycles, reduced testing costs, and increased team productivity.

3. Early Defect Detection

By covering a more comprehensive range of scenarios, ATCG tools are more likely to uncover defects earlier in development. This allows for quicker bug fixes and prevents defects from propagating to later stages, saving time and resources.

4. Continuous Integration and Delivery (CI/CD) Friendly

The automated nature of ATCG aligns perfectly with CI/CD pipelines. Test cases can be automatically generated and executed with each code commit, ensuring continuous quality monitoring and feedback throughout development.

5. Improved Software Reliability

With more comprehensive test coverage and earlier defect detection, ATCG contributes to the delivery of more robust and reliable software. This translates to a better user experience and fewer post-release issues.

Modern AI tools can now generate test code, suggest edge cases, and accelerate software testing workflows.

AI helps software engineers create faster, smarter, and more reliable tests.:

<https://www.youtube.com/watch?v=li4iByza908>

<https://www.youtube.com/watch?v=MUM5rQ5wji4>

Verification and Validation in **Agile**

- **Agile software development** emphasizes **iterative development**, **customer collaboration**, and **flexible response to change**. Agile integrates V&V **continuously** throughout the development lifecycle.
- **Verification and validation (V&V)** are essential processes in systems engineering to ensure that the system meets the requirements and expectations of the stakeholders.
- However, traditional **V&V** methods can be time-consuming, costly, and rigid, especially in complex and dynamic environments.
- **Agile methods**, on the other hand, are based on iterative, incremental, and collaborative principles that can enhance the flexibility, responsiveness, and quality of **V&V** activities.



What are agile methods?

- **Agile methods** are a set of practices and values that focus on delivering working software or systems in short cycles, with frequent feedback, adaptation, and collaboration.
- It is not a specific methodology, but rather an approach that can be implemented through frameworks like Scrum, Kanban, or XP.
- It emphasizes customer satisfaction and value delivery, working software over comprehensive documentation, responding to change over following a plan, individuals and interactions over processes and tools, self-organizing and cross-functional teams, and continuous improvement and learning.

Key Agile Practices

TDD → test first

BDD → shared understanding

CI/CD → continuous testing

Test-First Practices (TDD/BDD)

Test-first means you **write tests before writing the actual code**. This may feel backward at first, but it ensures that your code always does what it's supposed to do.

Test-Driven Development (TDD)

- TDD is a software development approach where you write a **failing test first**, then write just enough code to make the test pass, and finally improve the code while keeping it working.

```
# Step 1: Write test
```

```
def test_addition():  
    assert add(2, 3) == 5
```

```
# Step 2: Write function to pass test
```

```
def add(a, b):  
    return a + b
```

Why use TDD?

Helps prevent bugs early.

Ensures all code is testable.

Encourages simple, modular design.

JUnit, pytest,.. Are tools for TDD

Behavior-Driven Development (BDD)

- BDD extends TDD by writing tests in **natural language** that describes how the software should behave — making it easier for non-programmers to understand.
- Gherkin Syntax (English-like format)

Feature: Addition

Scenario: Add two numbers

Given I have entered 2 and 3 into the calculator

When I press add

Then the result should be 5

Tools for BDD

Cucumber (Java, Ruby)

Behave (Python)

SpecFlow (.NET)

Why use BDD?

Improves **communication** between developers, testers, and business users.

Makes sure the software meets **business requirements**.

Tests act as **live documentation**.

Collaborative V&V

- Agile encourages **cross-functional teams** (developers, testers, ops).
- Verification and validation are no longer the sole responsibility of QA(Quality Assurance):
 - Developers write tests.
 - Testers pair with developers
 - Product owners validate features via BDD scenarios

Traditional vs Agile

Traditional V&V	Agile
After coding	Continuous
QA only	Whole team
Manual heavy	Automated
Late feedback	Instant feedback

How can agile methods improve V&V?

Instead of waiting until the end:

- Developers test small parts continuously
- Bugs are found earlier
- Problems are easier and cheaper to fix

Example:

- After adding a new **login feature**:
 - test immediately
 - fix immediately
 - continue development

How can agile methods improve V&V?

In Agile:

- developers
 - testers
 - product owners
- work together.

Testing is not only the QA team's responsibility anymore.

Result:

- Better communication
- Fewer misunderstandings
- Better software quality

How can agile methods improve V&V?

Fast Feedback

Agile uses short development cycles (sprints).

This allows:

- quick customer feedback
- fast improvements
- rapid bug fixing

Example

Customer says:

“This feature is confusing.”

Team can improve it in the next sprint.

Key Idea

“In Agile, testing is not a separate phase.
Testing is part of the entire development process.”